SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1 Case Study

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS: **Total Marks: 25**

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

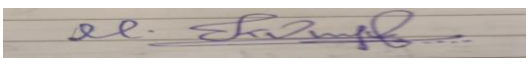
An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

I KUMARAN .M(192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: CNN for Automated Medical Image Classification

1. Suitable CNN Architecture

A suitable CNN architecture for classifying chest X-rays can be designed as follows:

Input X-ray Image → Preprocessing → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Global Average Pooling → Fully Connected Layer → Output

Example architecture:

Layer	Function
Input	Chest X-ray resized to 224 × 224 × 1
Conv Layer 1	32 filters, 3×3 kernel
ReLU	Adds non-linearity
Max Pooling	2×2, reduces image size
Conv Layer 2	64 filters, 3×3 kernel
ReLU	Feature learning
Max Pooling	2×2
Conv Layer 3	128 filters, 3×3 kernel
ReLU	Learns complex abnormalities
Global Average Pooling	Reduces feature maps
Fully Connected	Combines learned features
Output	Normal / Disease-affected

The X-ray images should first be resized and normalized. Data augmentation can also be applied during training.

2. Motivation and Role of CNN Components

CNNs are suitable because they can automatically learn important visual patterns from medical images without manually designing features.

Convolutional layers:
They scan the X-ray using small filters and extract features such as edges, textures, shapes, and abnormal regions.
Filters/Kernels:
A filter is a small matrix that moves across the image. Different filters learn different patterns. Early filters may detect edges, while deeper filters can detect more complex structures.
ReLU activation:
ReLU introduces non-linearity and helps the network learn complex relationships.
Pooling layers:
Pooling reduces the spatial dimensions of feature maps while retaining important information. Max pooling selects the maximum value from a small region. This reduces computation and provides some tolerance to small image variations.

Fully connected layers:
These layers combine the features learned by the convolutional layers and perform the final classification.
Output layer:
For two classes, the output can use sigmoid activation to produce the probability of the X-ray being disease-affected.

3. Parameter Sharing
Parameter sharing is one of the major advantages of CNNs.
In a fully connected network, every neuron may connect to every pixel. For example, a 224 × 224 grayscale image has:
224 × 224 = 50,176 pixels
Connecting these pixels to just 100 neurons would require approximately:
50,176 × 100 = 5,017,600 weights
This creates a very large number of parameters.
In a CNN, a 3 × 3 filter has only:

$3 \times 3 = 9$ weights + 1 bias = 10 parameters

The same filter is reused across different locations of the image. Therefore, the network can detect the same feature wherever it appears.

Advantages of parameter sharing:

- Reduces trainable parameters.
- Reduces memory and computation.
- Helps prevent overfitting.
- Makes CNNs effective for image data.

4. Regularization Techniques

Since the dataset may contain images with different quality and the model can easily overfit, regularization is important.

a) Dropout:

Randomly deactivates some neurons during training. This prevents the network from depending too heavily on particular neurons and improves generalization.

b) Data Augmentation:

Creates modified versions of training images using techniques such as:

- Small rotations
- Horizontal shifts
- Zooming
- Slight brightness/contrast changes
- Cropping

This increases the variety of training samples and makes the model more robust.

c) Batch Normalization:

Normalizes intermediate feature values during training. It can make training more stable and faster and can provide some regularization.

Other useful methods:

- Early stopping
- L2 regularization
- Learning-rate scheduling

For medical images, augmentation should be medically appropriate and should not create unrealistic X-ray appearances.

5. AlexNet vs ResNet

For this application, ResNet would generally be more appropriate than AlexNet.

Feature	AlexNet	ResNet
Network depth	Relatively shallow	Can be very deep
Feature learning	Good	Excellent
Skip connections	No	Yes
Vanishing-gradient problem	More likely in deeper networks	Reduced
Computational requirement	Lower	Higher
Image classification performance	Good	Generally better
Medical image suitability	Suitable for basic systems	Better for complex feature learning

Why choose ResNet?

Chest X-rays can contain subtle abnormalities that require learning complex hierarchical features. ResNet uses residual/skip connections, allowing information and gradients to flow more easily through deep networks. This makes it possible to train deeper models effectively.

A pretrained ResNet-18 or ResNet-50 can be fine-tuned using the chest X-ray dataset. Transfer learning can reduce training time and the amount of medical data required compared with training a deep network completely from scratch.

Although ResNet generally requires more computation than AlexNet, its improved feature extraction and classification performance make it the better choice when adequate computational resources are available.

Conclusion

A CNN can effectively classify chest X-rays by automatically learning features from simple edges to complex disease-related patterns. Convolution and parameter sharing make the network efficient, while pooling reduces dimensionality. Dropout, data augmentation, and batch normalization help reduce overfitting. For this medical classification task, ResNet is preferred over AlexNet because its deeper architecture and residual connections provide stronger feature learning and generally better classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

1. CNN Architecture Design

For an autonomous vehicle, the CNN should recognize objects such as cars, pedestrians, traffic signs, bicycles, and obstacles from camera images.

Suitable architecture:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Deep Convolution Layers → Global Average Pooling → Fully Connected Layer → Output Classes

Example:

- Input: 224 × 224 × 3 RGB image
- Conv Layer 1: 32 filters, 3×3
- Conv Layer 2: 64 filters, 3×3
- Conv Layer 3: 128 filters, 3×3
- Pooling: 2×2 max pooling
- Global Average Pooling
- Fully Connected Layer
- Output: Car, pedestrian, bicycle, sign, obstacle, etc.

For real autonomous driving, object detection is usually more useful than only classification because the system also needs to determine where each object is located.

2. Purpose of CNN Layers and Feature Learning

Input Layer:

Receives camera images and resizes/normalizes them before processing.

Convolution Layer:

Applies filters to detect important visual patterns. It reduces the need for manually designed features.

Activation Function (ReLU):

Introduces non-linearity, allowing the network to learn complex patterns.

Pooling Layer:
Reduces the size of feature maps, lowering computation while retaining important information.
Fully Connected/Output Layer:
Combines the extracted features and predicts the object category.
Progressive Feature Learning
CNN filters learn features in stages:
Early layers → edges, lines, corners
↓
Middle layers → textures, shapes, wheels, signs
↓
Deep layers → complete objects such as cars, pedestrians and bicycles.
Thus, deeper layers learn increasingly complex representations.

3. Importance of Parameter Sharing

In CNNs, the same filter weights are reused at different locations of an image.
For example, a filter that detects a vertical edge can detect that edge on the left, center, or right side of the image.
Benefits:

- Significantly reduces the number of trainable parameters.
- Reduces memory and computational requirements.
- Allows the same feature to be detected at different image locations.
- Improves translation-related feature detection.
- Makes CNNs more suitable for real-time image processing.

4. Overfitting and Regularization

Overfitting occurs when the CNN performs very well on training images but poorly on new road scenes.
Suitable methods include:

Data Augmentation:
Use realistic changes such as rotation, cropping, scaling, brightness changes, and horizontal flipping.
Dropout:
Randomly disables some neurons during training to reduce dependence on specific features.
Batch Normalization:
Normalizes activations and helps stabilize and speed up training.
Early Stopping:
Stops training when validation performance stops improving.
L2 Regularization:
Penalizes excessively large weights and helps improve generalization.
These techniques help the model perform reliably in different road and lighting conditions.

6. **AlexNet vs ResNet**

Feature	AlexNet	ResNet
Architecture	Basic deep CNN	CNN with residual/skip connections
Depth	Relatively shallow	Much deeper
Parameter efficiency	Less efficient	Better feature reuse
Training difficulty	Easier	More complex but easier to train deeply
Vanishing gradient	More likely	Greatly reduced
Feature learning	Good	Excellent
Accuracy	Good	Generally higher
Computation	Lower	Higher, depending on version

AlexNet:
Has a simpler architecture and requires fewer computational resources, making it easier to deploy.
ResNet:
Uses residual/skip connections, which allow information and gradients to pass through the network more effectively. This enables deeper networks to learn complex features without severe vanishing-gradient problems.

6. Recommended Architecture

ResNet is generally the better choice when accuracy and robust feature learning are the main priorities.
It can learn complex features needed to distinguish:

- Pedestrians from background objects
- Different types of vehicles
- Small traffic signs
- Objects at different distances
- Objects under varying lighting conditions

However, autonomous vehicles require real-time performance, so using a very large ResNet may be computationally expensive.
Therefore, a lightweight ResNet such as ResNet-18 or a compressed/optimized ResNet model would provide a good balance between accuracy, feature-learning capability, and computational cost.

Conclusion

For autonomous vehicle vision, ResNet is preferred over AlexNet because residual connections support deeper feature learning and generally provide better recognition performance. A lightweight ResNet can be optimized for real-time operation, making it suitable for detecting important road objects while maintaining a reasonable computational cost.